

Informática II

Programación Orientada a Objetos: Introducción

Profesor Dr. Paul Bustamante

Índice

- 1. Características de la POO**
- 2. Concepto de “clase” en C++**
- 3. Miembros de una clase**
- 4. Datos miembro de una clase**
- 5. Funciones miembro de una clase**
- 6. Objetos de una clase**
- 7. Control de un acceso a los datos miembros de una clase**
- 8. Organización de ficheros**
- 9. Pasos para crear un programa de POO**

1. Características de POO

Un **Objeto** es una encapsulación de un conjunto de **datos** (variables) y de **métodos** (funciones) para manipular a éstos.

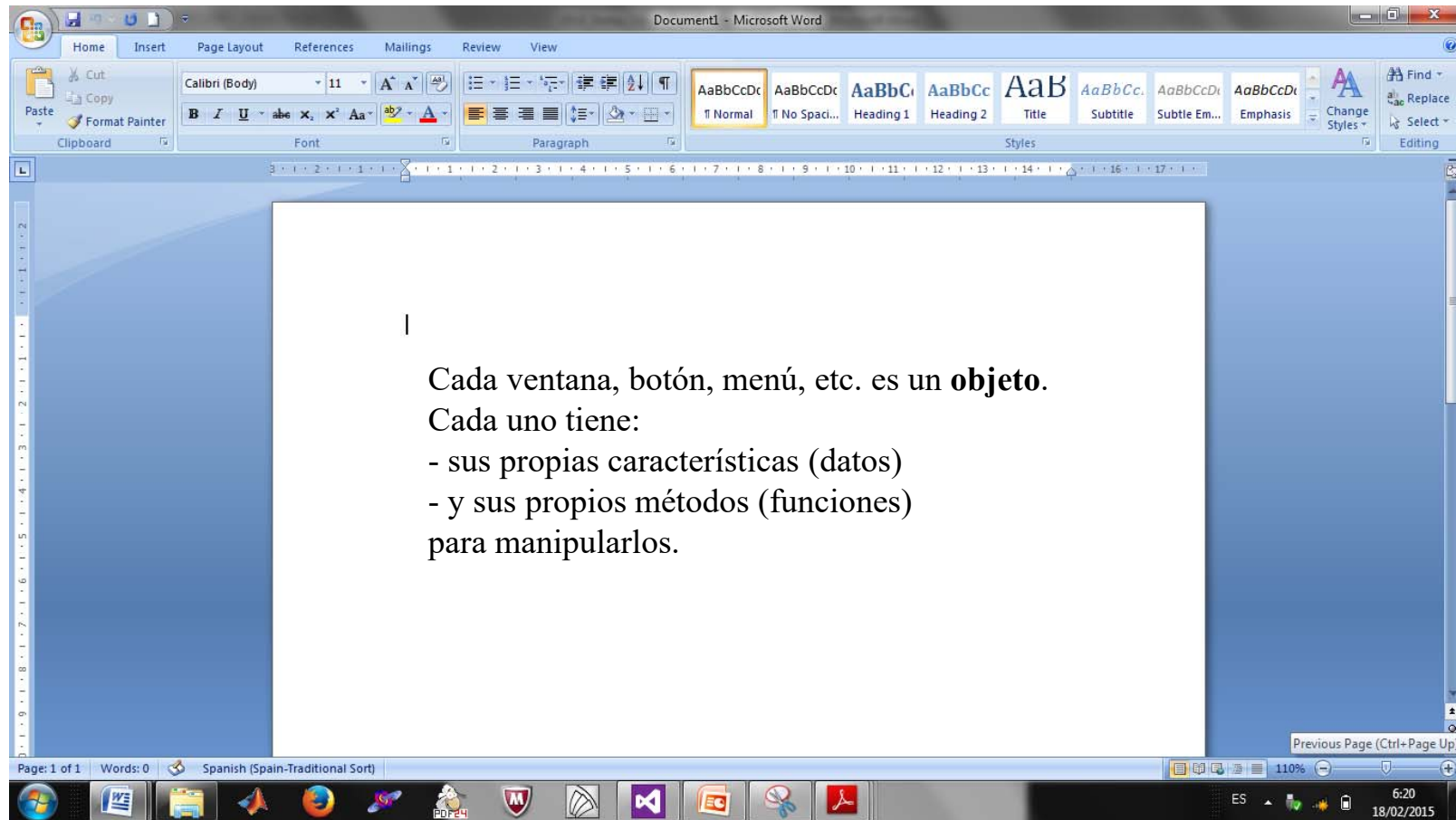
Características fundamentales de la Programación Orientada a Objetos (POO) son:

- **Abstracción:** Es la representación de las características esenciales de algo sin incluir los antecedentes o detalles irrelevantes.

La **clase** es una abstracción porque en ella se definen las propiedades y los atributos genéricos de un conjunto de objetos.

- **Encapsulación u ocultamiento de información:** Las variables y las funciones miembro de una clase pueden ser declaradas como **public**, **private** o **protected**. De esta forma se puede controlar el acceso a los miembros de la clase y evitar un uso inadecuado.
- **Herencia:** Es el mecanismo para compartir automáticamente métodos y atributos entre clases y subclases. Una clase puede derivar de otra, y en este caso hereda todas las variables y funciones miembro.
- **Polimorfismo:** Esta característica permite implementar múltiples formas de un mismo método, dependiendo cada una de ellas de la clase sobre la que se realice la implementación.

Windows: ejemplo de POO



2. Concepto de “clase” en C++

Una **Clase** es un **tipo de datos** definido por el usuario.

Básicamente, es una agrupación de datos (**variables**) y de funciones (**métodos**) que operan sobre esos datos.

La definición de una clase **consta de dos partes**:

1. **nombre de la clase**, precedido por la palabra reservada **class**.
2. **cuerpo de la clase**, encerrado entre llaves y seguido por (;):

```
class nombre_clase
{
    //cuerpo de la
    clase
};
```

```
class Rectangulo
{
    int base, altura;
public:
    void asignar_valores (int,int);
    void mostrar_valores (void);
    int area () {return base*altura;}
};
```

El **cuerpo de la clase** consta de:

- **Especificadores de acceso:** **public**, **protected** y **private**.
- **Atributos:** **datos miembro** de la clase (variables).
- **Métodos:** definiciones de **funciones miembro** de la clase.

Tanto las variables como los métodos pueden ser declarados **public**, **protected** y **private**, controlando de esta forma el acceso y evitando un uso inadecuado (**Encapsulación**).

Por omisión, los datos miembros de la clase son **privados**.

Por lo general, las definiciones de las clases se hacen en ficheros de cabecera (*.h), los cuales suelen llevar el nombre de la clase.

3. Miembros de una clase

Los **miembros** de una clase pueden ser **variables** de cualquier tipo y **funciones**.

- A las **variables** se las denomina: **datos miembro**.
- A las **funciones** se las denomina: **funciones miembro de la clase**.

```
class Rectangulo
{
    int base, altura;
public:
    void asignar_valores (int,int);
    void mostrar_valores (void);
    int area () {return base*altura;}
};
```

4. Datos miembro de una clase

La declaración de un **dato** miembro es igual que el de cualquier variable.

```
class Complejo
{
    public:                //especificador de acceso
        float real;        //Declaración correcta
        float imaginario = 0; //MAL, no se puede inicializar
};
```

Los datos miembro **no pueden ser inicializados** durante la declaración.

Si no se pone un **especificador de acceso** (como es nuestro ejemplo *public*), por defecto los datos miembro serán *private*.

En una clase cada dato miembro debe tener un **nombre único**.

También podemos declarar como datos miembro de una clase, **objetos** de otra clase, siendo necesario que ésta haya sido previamente definida.

```
class Datos
{
    Complejo c1;    //Declaración de un objeto de la clase Complejo
};
```

Para **acceder** a un dato miembro (siempre y cuando sea declarado como **público**) de una clase desde un objeto, se utilizará el **operador Punto (.)**. **Ejemplo:** *c1.real = 5.4;*

5. Funciones miembro de una clase

Las funciones miembro de una clase **definen las operaciones que se pueden realizar con sus datos miembro**.

Las funciones miembro también pueden ser **públicas o privadas**, lo cual se hace con los respectivos especificadores. Al igual que los datos miembro, si no se especifica serán privadas (*private*) por defecto.

Para **declarar una función miembro de una clase** se procede de la misma forma que para declarar una función cualquiera. Por **ejemplo** en el fichero “**complejo.h**”:

```
class Complejo
{
    private:
        float real, imaginario;
    public:      //funciones miembro públicas
        void Asignar (float x, float y);
};
```

```
void Complejo::Asignar(float x, float y)
{
    real = x; imaginario = y; //acceso a los
    datos                      //Más código
    ...
}
```

El cuerpo de una clase sólo contiene los prototipos de las funciones miembro (las declaraciones). **La definición de la función se hace en los ficheros fuente (*.cpp).**

Para **definir la función miembro en el fichero fuente** se utiliza el nombre de la clase seguido por el **operador de resolución de ámbito (::)**.

Ejemplo: en el fichero “**complejo.cpp**”:

5. Funciones miembro de una clase

Ejemplo:

En complejo.h

```
class Complejo
{
private:
    float real, imaginario;
public:
    void Asignar(float x, float y);
    void Print(void){
        cout << real << " + " << imaginario << "i" << endl;
    }
};
```

Las funciones miembro que tengan poco código también pueden ser definidas en el cuerpo de la clase.

En complejo.cpp:

```
void Complejo::Asignar(float x, float y)
{
    real = x; imaginario = y;
}
```

En nuestro ejemplo, la función **Print()** ha sido definida en el cuerpo de la clase, con lo cual ya no hay necesidad de hacerlo en el fichero fuente.

Dentro del cuerpo de la clase no hay necesidad de anteponer el nombre de la clase con el operador (::), como se hace generalmente en el fichero fuente, puesto que el nombre de la clase es conocido.

Para acceder a una función miembro (siempre y cuando sea declarada como *pública*) de una clase desde un objeto, se utilizará el **Operador Punto "."**.

Ejemplo: En el fichero **"complejo.cpp"**: `c1.Asignar( 5.4, 1.2);`

6. Objetos de una clase

Un **Objeto** es un ejemplar concreto de una clase. Las **clases** son como **tipos de variables**, mientras que los **objetos** son como **variables concretas** de un tipo determinado.

Los **objetos** constan de una estructura interna (los **datos**) y de una interfaz que permite manipular tal estructura (las **funciones**).

¿Cómo se construye un objeto de una clase? Igual que otra variable de un tipo predefinido, es decir, anteponiendo el **nombre de la clase** al nombre del objeto: **Complejo miComplejo**;

Un objeto **no se inicializa** como las variables:

Ejemplo: **Complejo miComplejo = 5 ;** **//Error**

Hay que recordar que **un objeto tiene una copia de todos los datos miembro de una clase**, entonces **¿cómo puedo inicializar esos datos miembro al momento de la creación del objeto?**:

1. Para solucionar este problema, C++ permite inicializar el objeto de una determinada clase en el momento de su declaración mediante un **constructor** definido por el usuario.
2. También existe otra forma de inicializar un objeto al momento de su declaración: **Complejo c1 = {1.5,-3.5};**

Nota: Esta forma es muy restrictiva y poco utilizada ya que exige que las variables miembro de la clase sean declaradas como públicas, que la clase no tenga constructores y que no sea derivada.

3. También se puede inicializar un objeto utilizando el **operador de asignación “=”**, utilizando para ello otro objeto que ya haya sido inicializado. Ejemplo: **comp1 = comp2;**

7. Control de acceso a los miembros de una clase

El concepto de clase incluye la idea de ocultación de datos, que, básicamente, consiste en que no se puedan acceder a los datos miembro directamente, sino que hay que hacerlo a través de las funciones miembro públicas de la clase.

Para controlar el acceso a los miembros (datos y funciones) de una clase, C++ provee de 3 especificadores:

1. **public**: un miembro declarado público es **accesible en cualquier parte del programa** donde el **objeto** de la clase en cuestión es accesible.
2. **private**: un miembro declarado privado puede ser accedido solamente **por las funciones miembro de su propia clase o por funciones amigas (friend)** de su clase. En cambio **no puede ser accedido** por funciones globales o por las funciones propias de una clase derivada (**herencia**).
3. **protected**: un miembro declarado protegido se comporta exactamente igual que uno privado para las funciones globales, pero **actúa como un miembro público** para las funciones miembro de una clase derivada.

```
#include "complejo.h"
void main(){
    Complejo c1;           //creación de un
    objeto
    c1.real = 4.5;          //error, real es private
    c1.Asignar(4.5,5.5);    //Correcto
    c1.Print();             //Bien, Print en public
}
```

```
//fichero complejo.cpp
#include "complejo.h"
void Complejo::Asignar(float x, float y)
{
    real = x; imaginario = y;
}
```

```
//fichero complejo.h
#include <iostream>
using namespace std;
class Complejo
{
    private:
        float real, imaginario;
    public:
        void Asignar(float x, float y);
        void Print(void){
            cout <<real<<" + "<<imaginario<<"i"<< endl;
        }
};
```

8. Organización de ficheros

Programa.cpp

- Contiene el programa principal con acciones sobre las clases

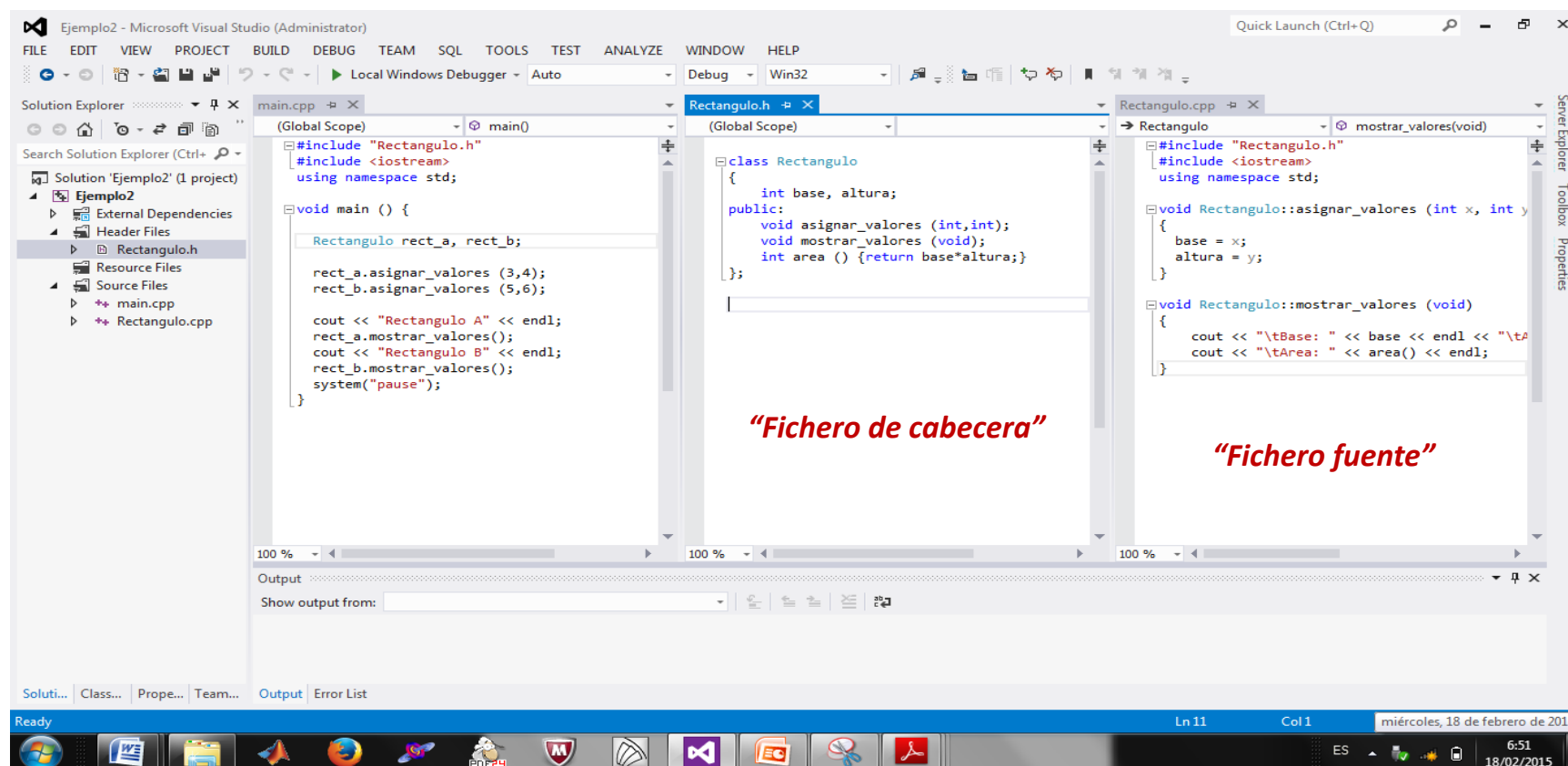
Nombre_clase.h

- Contiene la definición de la clase

Nombre_clase.cpp

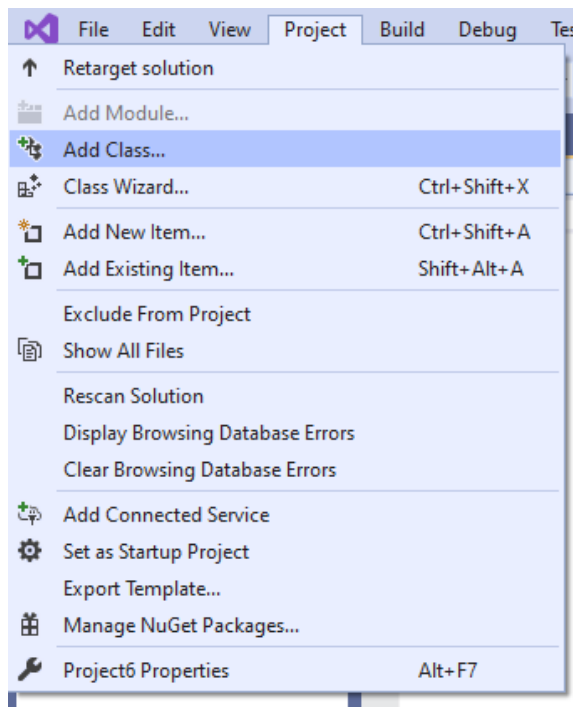
- Contienen la definición de las funciones y operadores de la clase

8. Organización de ficheros

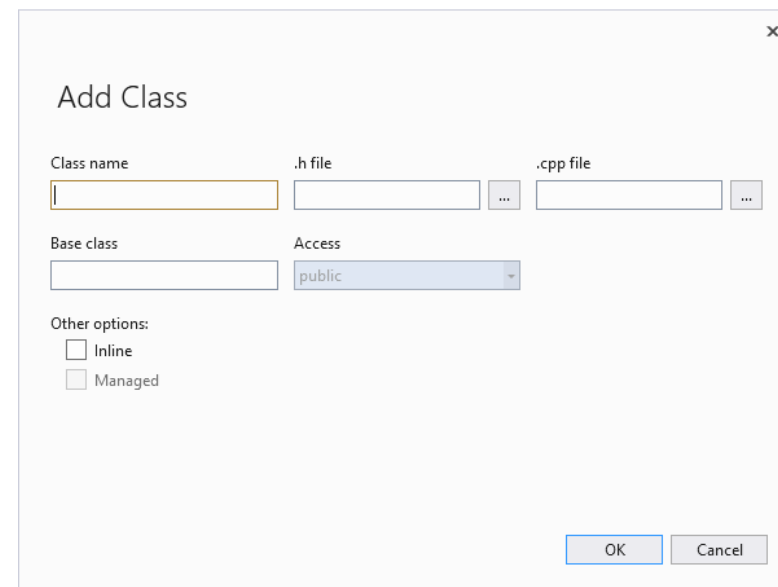


9. Pasos para crear un programa de POO

1. Crear un proyecto de C++ como se venía haciendo hasta la fecha, con el fichero **main.cpp**.
2. Añadir una nueva clase “**Proyecto/Add Class**” y elegir una clase tipo “**C++**” y clicar dos veces sobre dicho icono



3. Escribir el nombre de la clase y clicar “**OK**”



Ejemplo 1

Cálculo del módulo de un complejo:

```
#include <iostream>
using namespace std;
#include "complejo.h"

void main()
{
    Complejo c1,c2,c3;    // mi primer objeto
    double r,i;
    cout << "dar r e i: ";
    cin>>r>>i;

    c1.setdata(r,i); //c1: r=3 i=4

    double m = c1.modulo();
    cout << "Modulo de ";
    c1.prt();
    cout << " = " << m << endl;

    cout << "Fin" << endl;
}
```

Ejemplo 6-1

```
//fichero complejo.h
#include <iostream>
using namespace std;
class Complejo
{
    double r,i;//variables de la clase
public:
    void setdata(double r1, double i1);
    double modulo();
    void prt(){
        cout << r << "," << i << endl;
    }
};
```

```
//fichero complejo.cpp
#include "complejo.h"
double Complejo::modulo()
{
    double m = sqrt( r*r + i*i);
    return m;
}
void Complejo::setdata(double r1, double i1)
{
    r=r1; i=i1;
}
```

Ejemplo 2

Cálculo del área de un rectángulo:

A partir de la introducción de las medidas de dos rectángulos (base y altura):

- Mostrar dichas medidas por pantalla
- Calcular el área de cada uno de esos rectángulos
- Mostrar el área de cada uno de ellos por pantalla

```
#include "Rectangulo.h"
#include <iostream>
using namespace std;

void main () {
    Rectangulo rect_a, rect_b;

    rect_a.asignar_valores (3,4);
    rect_b.asignar_valores (5,6);

    cout << "Rectangulo A" << endl;
    rect_a.mostrar_valores();
    cout << "Rectangulo B" << endl;
    rect_b.mostrar_valores();
    system("pause");
}
```

Main.cpp

```
class Rectangulo
{
    int base, altura;
public:
    void asignar_valores (int,int);
    void mostrar_valores (void);
    int area () {return base*altura;}
};
```

Rectangulo.h

```
#include "Rectangulo.h"
#include <iostream>
using namespace std;

void Rectangulo::asignar_valores (int x, int y)
{
    base = x;
    altura = y;
}

void Rectangulo::mostrar_valores (void)
{
    cout << "\tBase: " << base << endl << "\tA"
    cout << "\tArea: " << area() << endl;
}
```

Rectangulo.cpp